Facultade de Informática
UNIVERSIDADE DA CORUÑA

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN ENXEÑARÍA DE COMPUTADORES

Sistema de captura y procesado de datos IMU en tiempo real para estimación de movimiento vertical

Estudante: Mateo Rivela Santos

Dirección: Dirección por asignar

A Coruña, febrero de 2026.

A mi familia, por su apoyo constante en cada etapa de este camino, y a todas las personas que me animaron a seguir aprendiendo cuando el resultado no salía a la primera.

Agradecimientos

Agradezco a mi entorno personal por la paciencia y el apoyo durante el desarrollo de este trabajo. Agradezco también al profesorado de la FIC por la base técnica y metodológica que ha hecho posible este proyecto, y a las personas que me ayudaron con pruebas, validaciones y revisión de resultados.

Resumen

Este trabajo presenta el desarrollo de un producto software móvil para captura y procesado de datos inerciales, con Android como plataforma ubicua principal. El sistema se diseña con dos modos de operación: modo autónomo (captura y procesado en el propio teléfono) y modo pantalla (visualización de resultados procesados por una plataforma externa como ESP32+MPU6050).

La solución implementada integra captura de sensores, normalización temporal, cálculo de métricas de oleaje y visualización en una misma aplicación, además de exportación de sesiones con parámetros y resultados. El algoritmo de procesado se utiliza como componente técnico del producto, no como fin en sí mismo.

Los resultados obtenidos muestran que la arquitectura de plataforma es viable y extensible, y permiten definir una comparativa homogénea entre Android y ESP32 para evaluar rendimiento, robustez y limitaciones en escenarios equivalentes.

Abstract

Este traballo presenta o desenvolvemento dun produto software móbil para captura e procesado de datos inerciais, con Android como plataforma ubicua principal. O sistema deseñouse con dous modos de operación: modo autónomo (captura e procesado no propio teléfono) e modo pantalla (visualización de resultados procesados por unha plataforma externa como ESP32+MPU6050).

A solución integra captura de sensores, normalización temporal, cálculo de métricas de ondada e visualización nunha mesma aplicación, ademais da exportación de sesións con parámetros e resultados. O algoritmo de procesado emprégase como compoñente técnico do produto, non como obxectivo final por si mesmo.

Os resultados mostran que a arquitectura de plataforma é viable e extensible, e permiten definir unha comparativa homoxénea entre Android e ESP32 para avaliar rendemento, robustez e limitacións en escenarios equivalentes.

Palabras clave:

- Unidad de medida inercial (IMU)
- Android
- Plataforma ubicua
- Producto software
- Procesado on-device
- Arquitectura móvil
- Comparativa Android-ESP32

Keywords:

- Unidade de medida inercial (IMU)
- Android
- Plataforma ubicua
- Produto software
- Procesado on-device
- Arquitectura móbil
- Comparativa Android-ESP32

Índice general

1 Introducción **1**

1.1 Contexto y motivación 1

1.2 Objetivos 1

1.2.1 Objetivo general 1

1.2.2 Objetivos específicos 2

1.3 Alcance y limitaciones 2

1.4 Metodología de desarrollo 2

1.5 Estructura de la memoria 3

2 Estado del arte y contexto tecnológico **4**

2.1 Plataforma móvil como nodo ubicuo 4

2.2 Procesado inercial para oleaje 4

2.3 Ventanas temporales y robustez 5

2.4 Comparativa de plataformas 5

3 Metodología, planificación y presupuesto **6**

3.1 Enfoque de trabajo 6

3.2 Fases del proyecto 6

3.3 Plan de pruebas y control de riesgos 7

3.4 Presupuesto orientativo 7

4 Diseño de la plataforma **8**

4.1 Arquitectura funcional 8

4.2 Diseño del bloque IMU 8

4.3 Diseño del flujo Android 9

4.4 Diseño de comparativa entre plataformas 9

5 Implementación del sistema **12**
5.1 Captura y normalización 12
5.2 Pipeline de procesado integrado 12
5.3 Visualización y exportación 13
5.4 Implementación de referencia para iteración técnica 13

6 Validación y evaluación **14**
6.1 Estrategia temporal de evaluación 14
6.2 Métricas de evaluación 14
6.3 Plantilla de comparativa Android vs ESP32+MPU6050 15
6.4 Criterios de aceptación 15

7 Conclusiones **16**
7.1 Conclusiones principales 16
7.2 Aportaciones 16
7.3 Trabajo futuro 17

A Material adicional **19**
A.1 Scripts principales 19
A.2 Comandos de referencia 19
A.2.1 Captura 19
A.2.2 Procesado rápido sin gráfica 19
A.2.3 Procesado paso a paso con validación temporal 19
A.3 Datos de prueba 20

Bibliografía **22**

Índice de figuras

4.1 Arquitectura funcional del TFG con Android como plataforma central. 8
4.2 Flujo Android: captura, procesado on-device, visualización y exportación. . . . 10
4.3 Flujo de comparativa entre plataformas con métricas unificadas. 11

5.1 Visualización por etapas del pipeline utilizada en la fase de validación. 13

Índice de cuadros

6.1 Plantilla de comparación Android vs ESP32+MPU6050 con protocolo homogéneo. 15

Capítulo 1

Introducción

Este trabajo fin de grado se centra en una idea principal: usar el teléfono móvil como plataforma ubicua para capturar y procesar datos inerciales, y demostrar su viabilidad en un caso de uso de caracterización de oleaje. El valor del sistema no está en un sensor concreto, sino en el producto software: su arquitectura de plataforma, su operación en distintos modos y su capacidad de despliegue práctico.

1.1 Contexto y motivación

La señal inercial medida por un dispositivo móvil combina aceleración útil, gravedad, ruido, deriva y cambios de orientación, lo que dificulta una interpretación directa [1]. Para transformar estas medidas en métricas útiles de estado de mar, se requiere un pipeline de procesado robusto y consistente.

En este trabajo se adopta como método principal un enfoque tipo OpenMetBuoy: estimación espectral mediante Welch, conversión de espectro de aceleración a elevación y cálculo de métricas de oleaje (H_s, T_z, T_c). La implementación se orienta a ejecución *on-device* en Android, con una fase posterior de comparación frente a ESP32+MPU6050.

1.2 Objetivos

1.2.1 Objetivo general

Diseñar e implementar una plataforma móvil capaz de capturar y procesar datos inerciales para estimación de movimiento vertical de olas, con salida reproducible y comparable entre plataformas.

1.2.2 Objetivos específicos

- Implementar en Android un modo autónomo de captura y procesado local.
- Estandarizar formato de sesión y exportación (CSV/JSON) con metadatos de adquisición.
- Aplicar un pipeline espectral tipo OpenMetBuoy para estimar H_s, T_z y T_c.
- Incluir visualización de olas $h(t)$, métricas y estado de validez de ventana.
- Implementar un segundo modo en Android como pantalla de resultados de ESP32+MPU6050.
- Definir una comparativa experimental homogénea Android vs ESP32 con métricas comunes.

1.3 Alcance y limitaciones

El alcance de la memoria es la estimación de movimiento vertical relativo en superficie, no navegación inercial completa. No se aborda estimación absoluta de orientación/posición con fusión avanzada magnetómetro–GNSS, por su mayor complejidad [2].

Las limitaciones principales consideradas son:

- Sensibilidad de la estimación a orientación y calidad de medida.
- Necesidad de ventanas largas (10 a 20 min) para resultados espectrales robustos.
- Menor fiabilidad del modo rápido en ventanas cortas (30 s), útil solo como diagnóstico.
- Diferencias de tasa efectiva de muestreo entre plataformas.

1.4 Metodología de desarrollo

La metodología aplicada es incremental y orientada a validación:

1. Definición de arquitectura funcional con Android como núcleo.
2. Implementación y pruebas del pipeline de procesado en entorno reproducible.
3. Validación de estabilidad del método con ventanas largas y análisis de límites en ventanas cortas.
4. Integración del modo Android display para consumir resultados externos.
5. Diseño del protocolo de comparación Android vs ESP32+MPU6050.

1.5 Estructura de la memoria

La memoria se organiza de la siguiente forma:

- En el Capítulo 2 se resume el estado del arte estrictamente necesario para justificar decisiones de producto.
- En el Capítulo 3 se describe la metodología de desarrollo, la planificación y el presupuesto de trabajo.
- En el Capítulo 4 se define la arquitectura funcional de la plataforma y sus módulos, incluyendo IMU y modos de operación.
- En el Capítulo 5 se documenta la implementación técnica del producto y el formato de datos.
- En el Capítulo 6 se presenta el plan de pruebas y la validación experimental Android vs ESP32+MPU6050.
- En el Capítulo 7 se recogen conclusiones, aportaciones y trabajo futuro.
- En el Apéndice A se incluye material complementario.

Capítulo 2

Estado del arte y contexto tecnológico

Este capítulo recoge únicamente el contexto técnico necesario para justificar la solución propuesta, manteniendo el foco en desarrollo de producto.

2.1 Plataforma móvil como nodo ubicuo

El uso de smartphone como nodo de captura aporta disponibilidad, conectividad y capacidad de cómputo en un único dispositivo. Para este TFG, Android se adopta como plataforma principal por:

- acceso directo a sensores inerciales integrados;
- ecosistema de desarrollo maduro y despliegue rápido;
- capacidades locales de almacenamiento, visualización y exportación.

2.2 Procesado inercial para oleaje

La estimación de estado de mar a partir de IMU se aborda en la literatura con técnicas espectrales. En este trabajo se adopta un enfoque tipo OpenMetBuoy por su trazabilidad y aplicabilidad práctica, evitando desarrollar un algoritmo propio desde cero.

El objetivo del TFG no es investigar un nuevo método, sino integrar un método existente dentro de una plataforma usable y evaluable.

2.3 Ventanas temporales y robustez

Una conclusión técnica recurrente en trabajos y sistemas similares es la necesidad de ventanas de adquisición suficientemente largas para estabilizar métricas espectrales. Por ello, este trabajo diferencia dos modos:

- modo robusto con ventanas de 10 a 20 minutos;
- modo rápido para diagnóstico local, no equivalente al resultado robusto.

2.4 Comparativa de plataformas

Además del modo Android autónomo, se plantea una segunda fase con ESP32+MPU6050 para contrastar viabilidad en hardware dedicado. La comparación se define con mismo protocolo físico y mismo formato de salida, para asegurar una evaluación justa de resultados.

Capítulo 3

Metodología, planificación y presupuesto

La ejecución del TFG se organiza como un desarrollo incremental de producto, con validación continua sobre datos reales y simulados.

3.1 Enfoque de trabajo

Se adopta una estrategia por bloques funcionales:

1. Captura IMU y normalización temporal.
2. Procesado espectral on-device.
3. Visualización y exportación de sesión.
4. Modo display para integración con ESP32.
5. Validación comparativa final entre plataformas.

3.2 Fases del proyecto

- **Fase 1.** Definición de arquitectura del producto y formato común de datos.
- **Fase 2.** Prototipado de captura/procesado con scripts de referencia para reducir riesgo técnico.
- **Fase 3.** Integración on-device en Android y diseño de interfaz operativa.
- **Fase 4.** Integración ESP32+MPU6050 y modo Android display.
- **Fase 5.** Campaña comparativa y elaboración de conclusiones.

3.3 Plan de pruebas y control de riesgos

Los principales riesgos y su mitigación son:

- Ventanas insuficientes para estabilidad espectral: imponer ventana robusta mínima.
- Variabilidad de frecuencia de muestreo: medir f_s real por sesión y registrar jitter.
- Diferencias de formato Android/ESP32: fijar esquema de salida común desde el inicio.
- Deriva o ruido excesivo: incluir validación de calidad y descarte de sesiones no válidas.

3.4 Presupuesto orientativo

El trabajo prioriza bajo coste de hardware:

- teléfono Android: plataforma principal;
- ESP32 con LoRa: disponible en laboratorio personal;
- IMU MPU6050: disponible para fase comparativa.

El coste principal del proyecto se concentra en tiempo de desarrollo, integración y validación experimental.

Capítulo 4

Diseño de la plataforma

El diseño se centra en Android como núcleo del producto, con dos modos operativos y módulos desacoplados de captura, procesado, visualización y exportación.

4.1 Arquitectura funcional

La arquitectura global se divide en:

- **Modo Android autónomo**: captura y procesa en el propio teléfono.
- **Modo Android display**: recibe resultados desde ESP32+MPU6050 y los presenta de forma homogénea.

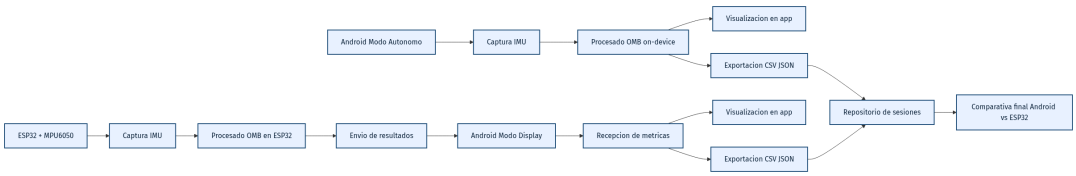

Figura 4.1: Arquitectura funcional del TFG con Android como plataforma central.

4.2 Diseño del bloque IMU

El bloque IMU se modela como un módulo de producto, no como experimento aislado:

- adquisición de `accelerometer` y `gyroscope`;
- normalización de tiempo y estimación de frecuencia efectiva;
- serialización en formato común para procesado y exportación.

Este diseño permite reutilizar el mismo contrato de datos tanto en Android como en ESP32.

4.3 Diseño del flujo Android

Cada sesión incluye:

- series temporales inerciales;
- metadatos de dispositivo y parámetros de cálculo;
- métricas finales (H_s, T_z, T_c, f_s real, indicadores de calidad).

4.4 Diseño de comparativa entre plataformas

La comparativa se diseña para medir diferencias de estabilidad, ruido, latencia y consumo bajo un mismo protocolo de prueba.

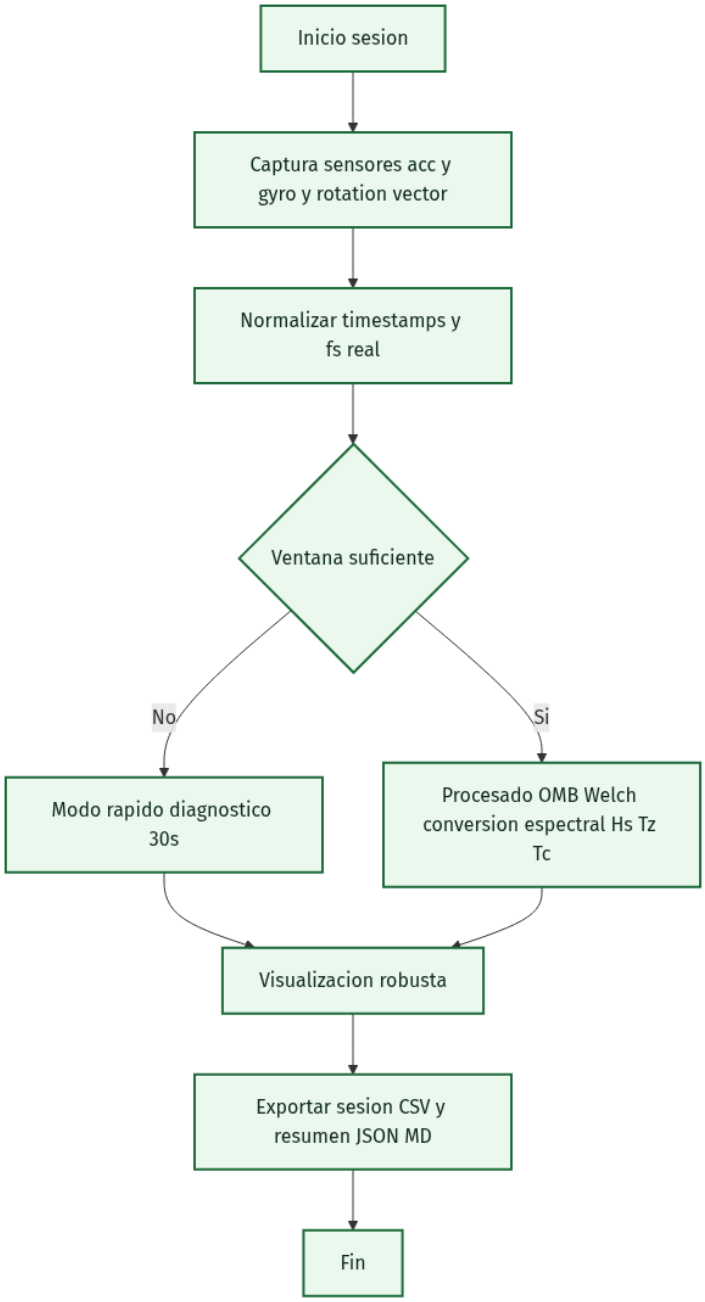

Figura 4.2: Flujo Android: captura, procesado on-device, visualización y exportación.

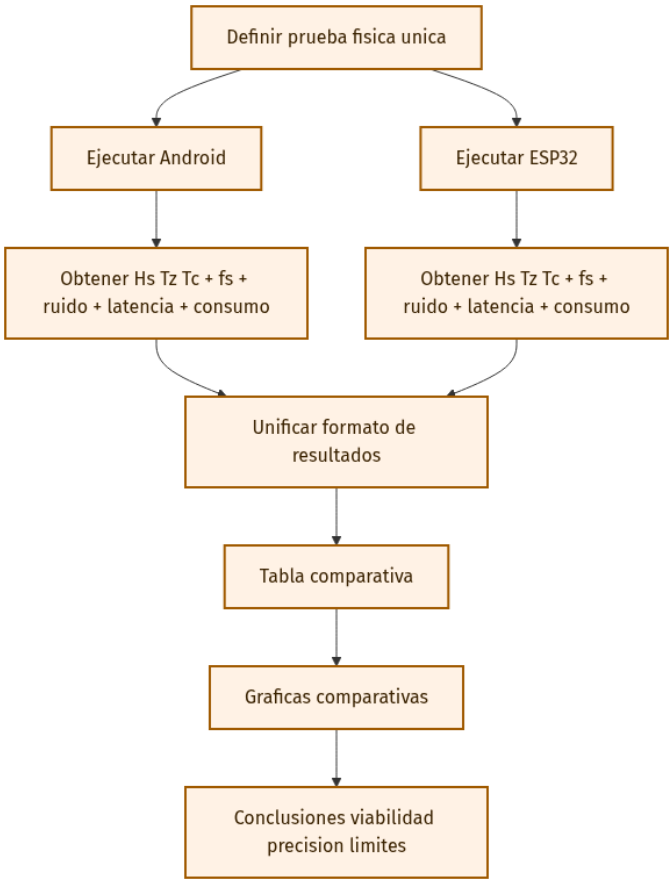

Figura 4.3: Flujo de comparativa entre plataformas con métricas unificadas.

Capítulo 5

Implementación del sistema

En este capítulo se describe la implementación técnica del producto y las decisiones adoptadas para garantizar reproducibilidad.

5.1 Captura y normalización

La captura en Android registra aceleración y velocidad angular con timestamp. La normalización posterior calcula la frecuencia efectiva de muestreo y corrige irregularidades temporales.

5.2 Pipeline de procesado integrado

El método implementado sigue una secuencia espectral tipo OpenMetBuoy:

1. estimación de f_s real;
2. separación de gravedad para obtener aceleración dinámica vertical;
3. PSD de aceleración mediante Welch;
4. conversión a elevación:
$$S_\eta(f) = \frac{S_a(f)}{(2\pi f)^4}$$
5. cálculo de momentos y métricas:
$$H_s = 4\sqrt{m_0}, \qquad T_z = \frac{1}{\sqrt{m_2/m_0}}, \qquad T_c = \frac{1}{\sqrt{m_4/m_2}}$$

Se trata de un método adoptado e integrado en la plataforma, no de una propuesta algorítmica original [3].

5.3 Visualización y exportación

La aplicación muestra:

- evolución temporal de olas $h(t)$;
- panel de métricas actuales;
- indicador de validez de ventana para resultado robusto.

La exportación de sesión incluye datos crudos, datos procesados, métricas y parámetros de ejecución.

5.4 Implementación de referencia para iteración técnica

Durante el desarrollo se empleó un entorno de scripts Python para acelerar pruebas de parámetros y depuración del pipeline. Esta capa de soporte queda documentada como herramienta de ingeniería, mientras que el producto objetivo es la ejecución en Android.

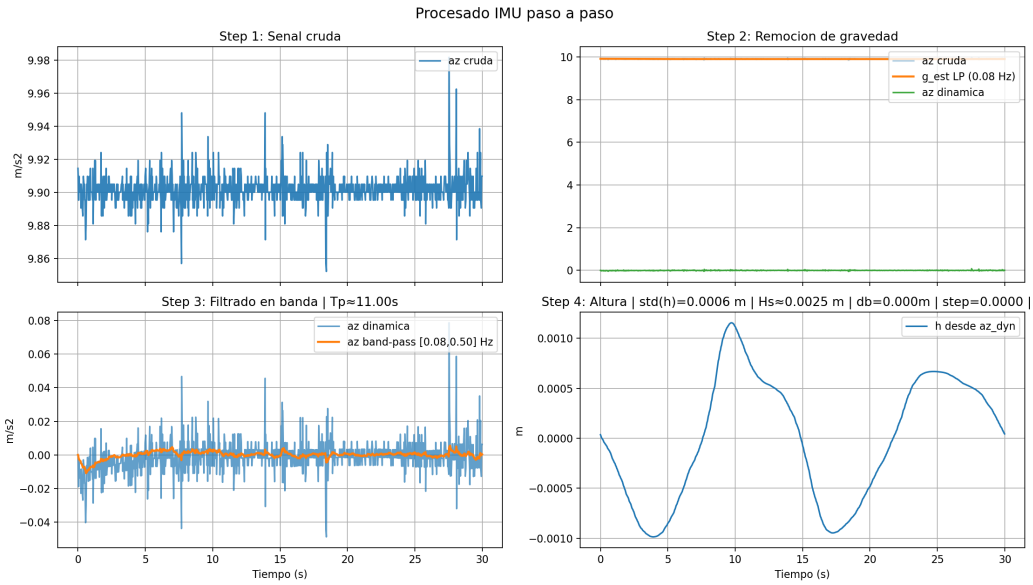

Figura 5.1: Visualización por etapas del pipeline utilizada en la fase de validación.

La base de captura de sensores en Android se apoyó en la documentación oficial [4].

Capítulo 6

Validación y evaluación

La validación se enfoca en comprobar la viabilidad del producto en condiciones de uso real y en preparar una comparación defendible entre plataformas.

6.1 Estrategia temporal de evaluación

La validación distingue dos escenarios:

- **Ventana robusta (10–20 min)**: cálculo principal para métricas finales.
- **Ventana corta ($\sim$30 s)**: diagnóstico rápido de señal y funcionamiento.

Los resultados robustos del TFG se reportan sobre ventana larga.

6.2 Métricas de evaluación

Las métricas principales son:

- H_s, T_z, T_c y su estabilidad temporal;
- frecuencia efectiva de muestreo y jitter;
- latencia de cálculo y visualización;
- consumo por sesión;
- indicadores de calidad de señal.

Métrica	Android	ESP32+MPU6050	Criterio de evaluación
Frecuencia efectiva (Hz)	Medida en sesión	Medida en sesión	Estabilidad temporal y jit
H_s, T_z, T_c	Serie temporal	Serie temporal	Concordancia entre pla formas
Ruido residual	Estimado tras procesado	Estimado tras procesado	Robustez de la estimació
Latencia	On-device / display	Envío + display	Respuesta en uso real
Consumo energético	Por sesión de prueba	Por sesión de prueba	Viabilidad operativa

Cuadro 6.1: Plantilla de comparación Android vs ESP32+MPU6050 con protocolo homogéneo.

6.3 Plantilla de comparativa Android vs ESP32+MPU6050

6.4 Criterios de aceptación

Se considerará la plataforma validada cuando:

- complete el flujo captura-procesado-visualización-exportación sin intervención externa;
- mantenga estabilidad de métricas en sesiones largas repetidas;
- documente límites de precisión y condiciones de no validez;
- permita comparar Android y ESP32 con resultados trazables.

Capítulo 7

Conclusiones

El trabajo queda reorientado y consolidado en torno a una meta clara: demostrar que un teléfono móvil puede actuar como plataforma ubicua para captura y procesado inercial, con una metodología reproducible y extensible a otras plataformas.

7.1 Conclusiones principales

- El enfoque de plataforma Android como núcleo funcional es viable y coherente con los objetivos del TFG.
- El método espectral tipo OpenMetBuoy proporciona una base técnica sólida para estimar H_s, T_z y T_c.
- La estabilidad del método depende críticamente de la ventana temporal: las ventanas largas (10–20 min) son el modo principal de estimación robusta.
- El modo rápido en ventanas cortas es útil para diagnóstico operativo, pero no debe considerarse equivalente al cálculo robusto.
- La definición de un flujo común Android/ESP32 permite una comparativa final técnicamente defendible.

7.2 Aportaciones

Las aportaciones del trabajo en su estado actual son:

- Definición de arquitectura dual en Android (modo autónomo y modo display).
- Formalización de pipeline de procesado y métricas de salida unificadas.
- Esquemas funcionales y plantilla de evaluación comparativa entre plataformas.

- Entorno reproducible para validación previa de decisiones de diseño.

7.3 Trabajo futuro

Las siguientes líneas continúan directamente el flujo definido:

- Implementar el procesado on-device completo en Android como camino principal.
- Implementar el flujo equivalente en ESP32+MPU6050 con mismo formato de resultados.
- Ejecutar campaña de pruebas comparativas con protocolo físico común.
- Incorporar estimación de calidad de señal y criterios automáticos de validez por sesión.
- Automatizar exportación de informe por sesión para análisis y memoria final.

Apéndices

Apéndice A

Material adicional

En este apéndice se recoge material de apoyo para reproducir el entorno experimental y el flujo de trabajo técnico del proyecto.

A.1 Scripts principales

- `capture_http_imu.py`: recepción HTTP y guardado de sesiones.
- `process_imu_session.py`: procesado offline con métricas y visualización.
- `process_imu_step_by_step.py`: visualización por etapas del pipeline.

A.2 Comandos de referencia

A.2.1 Captura

```
python3 capture_http_imu.py --host 0.0.0.0 --port 8001 --seconds 30
```

A.2.2 Procesado rápido sin gráfica

```
python3 process_imu_session.py session_YYYYMMDD_HHMMSS_30s.csv
    --no-plot
```

A.2.3 Procesado paso a paso con validación temporal

```
python3 process_imu_step_by_step.py session_YYYYMMDD_HHMMSS_30s.csv
    \
  --mode step4 --g-fc 0.03 --fmin 0.02 --fmax 1.20 \
  --height-min-change 0.00005 --height-step-min 0.000001
    --confirm-samples 6
```

A.3 Datos de prueba

Se trabajó con múltiples sesiones de 30 segundos en formato CSV, incluyendo capturas de reposo y capturas con movimiento vertical manual. La bitácora de resultados y ajustes se mantiene en `INFORME_IMU.md`.

Bibliografía

[1] D. H. Titterton and J. L. Weston, *Strapdown Inertial Navigation Technology*, 2nd ed. The Institution of Engineering and Technology, 2004.

[2] J. A. Farrell, *Aided Navigation: GPS with High Rate Sensors.* McGraw-Hill, 2008.

[3] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Prentice Hall, 2010.

[4] Android Developers, "Sensor overview," https://developer.android.com/guide/topics/sensors/sensors_overview, 2026, accedido el 18 de febrero de 2026.